Jack Cashman

cashman3@purdue.edu | jackfcashman@gmail.com | +1 (516) 659-2087
cashmanjack.github.io | Google Scholar | LinkedIn | GitHub

Education

Purdue University **August 2026 – Present**
Ph.D. in Electrical and Computer Engineering West Lafayette, IN

University of Rochester **August 2022 – May 2026**
Bachelor of Science, Honors in Computer Science Rochester, NY

Relevant Coursework: Advanced Compiler Optimizations, Machine Learning Systems, Programming Language Design, Computer Organization, Computer Architecture, Operating Systems.

Awards & Fellowships

NSF Graduate Research Fellowship (GRFP) **2026**
National Science Foundation

Research Experience

Graduate Research Assistant **August 2026 – Present**
Purdue University, advised by Prof. Milind Kulkarni

- Built a GPU engine for multi vector late interaction retrieval, restructuring candidate generation with a single radix sort/reduce and a fused tensor core rerank with about 11× lower latency than PLAID
- Evaluated GPU RT cores for retrieval candidate generation
- Characterizing when RT hardware outperforms SM's, working toward a predictive cost model
- Developing a Rust based DSL that lowers ray tracing traversal primitives

Honors Thesis **August 2025 – April 2026**
University of Rochester, Dept. of Computer Science

- Developed a novel sparse matrix reordering solution that uses analytical cache modeling, rather than traditional graph theory approaches, to drive reordering decisions.
- Designed an analytical cache model that directly targets performance factors including cache capacity, associativity, latency, and replacement policy, prefetchers, thread count, and more.
- Performed genetic search over analytical prediction and validated with hardware counters.
- Parallelized reordering search and analytical modeling for improved reordering speed.

Research Assistant Under Prof. Ponnuswamy Sadayappan **May 2025 – July 2025**
University of Utah

- Investigated how sparse matrix reordering schemes affect cache hierarchy utilization and Sparse Matrix Vector Multiplication (SpMV) performance.
- Reduced error of a machine learning model for pairwise ranking of data layouts based on predicted L1D, L2, and L3 cache misses, addressing the fragility of traditional reordering heuristics.
- Implemented and validated hardware counter frameworks (**VTune, PAPI, LIKWID, Perf**) to capture precise cache performance data for model training and empirical validation.
- Conducted controlled experiments by disabling hardware prefetching and varying compiler optimization levels to isolate performance effects and align empirical results with model predictions.

Research Assistant Under Prof. Chen Ding **May 2024 – July 2024**
University of Rochester

- **Loop Asymptotic Locality Analysis:** Developed a symbolic tool to model reuse distance distribution of affine programs, reducing analysis time from doubly exponential to linear.

– **Zipfian Caching for Web Workloads:** Designed and evaluated caching policies to outperform LRU on dynamic Zipfian web workloads, achieving a faster convergence to the optimal hit rate.
– **LLM Text Detection:** Used reuse distance for varying N gram text representation to classify text as human vs. AI generated, building an N gram trace generator for feature extraction.

Teaching Experience

Teaching Assistant **Aug 2024 – April 2026**
University of Rochester

– Served as a TA for Computer Organization and Programming Language Design and Implementation
– Led study sessions and office hours to help undergraduate students understand core systems concepts, including memory hierarchy, instruction sets, and digital logic.
– Wrote, graded, and proctored exams.
– Graded student projects and provided feedback on assembly language and C programming assignments.

Selected Projects

Latency Based Tiling **Spring 2025**
Course Research Project, Honors Undergraduate Problem Seminar

– Built a triangle loop micro benchmark for approximating L1D/L2/L3 cache capacities.
– Leveraged the cache approximation for polyhedral program tiling to produce a 21% performance speedup with effectively no compilation overhead unlike auto-tuning.

Mid-End Compiler Optimization Framework **Spring 2025**
Course Project, Advanced Compilers (Software Analysis & Improvement)

– Built a compiler optimization framework from scratch, operating on an intermediate representation.
– Implemented classic and advanced optimizations including dead code elimination, partial redundancy elimination, and constant propagation using abstract interpretation.

Tiny-QMOE **Fall 2024**
Course Research Project, Machine Learning Systems

– Engineered a quantization and dictionary-based compression pipeline to enable inference of large language models (LLMs) on memory-constrained devices (e.g., mobile phones).
– Achieved **23–35x compression** on LLaMA models with minimal loss on MMLU and ARC benchmarks.
– Gained experience with GPU memory hierarchy, compression algorithms, and LLM benchmarking.

Publications

– **Preprint:** Jack Cashman. *Latency Based Tiling.* arXiv preprint arXiv:2510.15912, 2025.
– **Preprint:** Jack Cashman, Jiaqi Nie. *Tiny-QMoE.* arXiv preprint arXiv:2509.22951, 2025.

Technical Skills

Languages: C++, Python, Rust, C, RISC-V Assembly, x86-64 Assembly, Coq, CUDA

Developer Tools: Git, Linux/Unix Shell, OptiX, OWL, VTune, PAPI, LIKWID, Perf, GDB

Areas of Interest: Compiler Optimization, Locality, Cache Modeling, Computer Microarchitecture, Hardware Accelerators, Domain Specific Acceleration, Type Systems/Safety, Machine Verified Proofs